Day 4 — Advanced Java Programming Concepts

Simple Questions, Answers, Code Examples, and a Student Data Processing Application

1. What is an array?

An array stores a fixed number of values of the same type. Each element is accessed by an index starting at 0.

```
int[] marks = {75, 90, 82};
System.out.println(marks[0]); // 75
```

2. What is a String?

A String stores text. Strings are immutable in Java, meaning a String object cannot be changed after it is created. Methods such as length(), equals(), and toLowerCase() are commonly used.

```
String name = "Asha";
System.out.println(name.length());       // 4
System.out.println(name.equals("Asha")); // true
```

3. What is the Java Collections Framework?

The Collections Framework provides classes and interfaces for storing and working with groups of objects. Common types include List, Set, and Map. ArrayList and HashMap are frequently used implementations.

4. What is an ArrayList?

ArrayList is a resizable list. It keeps items in order, allows duplicates, and can grow or shrink as needed.

```
import java.util.ArrayList;

ArrayList<String> names = new ArrayList<>();
names.add("Asha");
names.add("Ravi");
names.add("Asha");
System.out.println(names.get(0)); // Asha
```

5. What is a HashMap?

HashMap stores key-value pairs. Each key must be unique. It is useful when you want to find a record quickly using a key such as an ID.

```
import java.util.HashMap;

HashMap<Integer, String> students = new HashMap<>();
students.put(101, "Asha");
students.put(102, "Ravi");
System.out.println(students.get(101)); // Asha
```

6. What is exception handling?

Exception handling helps a program respond to errors instead of stopping unexpectedly. Java uses try, catch, finally, throw, and throws. Catch specific exceptions where possible.

```
try {
    int number = Integer.parseInt("abc");
} catch (NumberFormatException e) {
    System.out.println("Please enter a valid integer.");
}
```

7. What is file handling?

File handling lets a program save data to and read data from files. Java provides classes such as Files, Path, BufferedReader, and BufferedWriter. Use try-with-resources to close resources safely.

```
import java.nio.file.Files;
import java.nio.file.Path;
import java.io.IOException;

try {
    Files.writeString(Path.of("notes.txt"), "Hello Java");
    String text = Files.readString(Path.of("notes.txt"));
    System.out.println(text);
} catch (IOException e) {
    System.out.println("File operation failed.");
}
```

8. What are Java Streams?

Streams provide a way to process data from a collection using operations such as filter, map, and sorted. A stream does not itself store data; it processes data from a source.

```
import java.util.Arrays;
import java.util.List;

List<Integer> marks = Arrays.asList(70, 90, 55, 82);
marks.stream()
     .filter(m -> m >= 75)
     .sorted()
     .forEach(System.out::println);
```

9. How can you search for a record by ID or name?

For an ID, a HashMap is convenient because the ID can be used as a key. For searching by name in an ArrayList, loop through the records and compare names using equalsIgnoreCase().

```
// Search by ID:
Student found = studentsById.get(101);

// Search by name in a list:
for (Student s : studentList) {
    if (s.getName().equalsIgnoreCase("Asha")) {
        System.out.println(s.getName());
    }
}
```

10. How do you sort records by name or marks?

Use Comparator with sort() to order records by a field. The example assumes Student has getName() and getMarks() methods.

```
studentList.sort(
    java.util.Comparator.comparing(Student::getName)
);

studentList.sort(
    java.util.Comparator.comparingDouble(Student::getMarks)
);
```

11. Mini Project: Student Data Processing Application

This console example stores student records in an ArrayList and a HashMap, searches by ID, sorts by name, and catches invalid numeric input. To keep the example simple, each record contains an ID, name, and marks.

```
import java.util.*;
import java.util.stream.Collectors;

class Student {
    private int id;
    private String name;
    private double marks;

    Student(int id, String name, double marks) {
        this.id = id;
        this.name = name;
        this.marks = marks;
    }

    int getId() { return id; }
    String getName() { return name; }
    double getMarks() { return marks; }

    public String toString() {
        return id + " | " + name + " | " + marks;
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        List<Student> list = new ArrayList<>();
        Map<Integer, Student> byId = new HashMap<>();

        try {
            System.out.print("How many students? ");
            int count = Integer.parseInt(sc.nextLine());

            for (int i = 0; i < count; i++) {
                System.out.print("ID: ");
                int id = Integer.parseInt(sc.nextLine());

                System.out.print("Name: ");
                String name = sc.nextLine();

                System.out.print("Marks: ");
                double marks = Double.parseDouble(sc.nextLine());

                Student s = new Student(id, name, marks);
                list.add(s);
                byId.put(id, s);
            }

            System.out.print("Search student ID: ");
            int searchId = Integer.parseInt(sc.nextLine());
            Student found = byId.get(searchId);
            System.out.println(found == null ? "Not found" : found);

            list.sort(Comparator.comparing(Student::getName,
                    String.CASE_INSENSITIVE_ORDER));
            System.out.println("Sorted by name:");
            list.forEach(System.out::println);

            System.out.println("Students with marks >= 75:");
            list.stream()
                .filter(s -> s.getMarks() >= 75)
                .forEach(System.out::println);

        } catch (NumberFormatException e) {
            System.out.println("Invalid input: enter numbers for ID, count, and marks.");
        } finally {
            sc.close();
        }
    }
```

```
}
```

Example input and output (shortened)

```
How many students? 2
ID: 101
Name: Asha
Marks: 88
ID: 102
Name: Ravi
Marks: 72
Search student ID: 101
101 | Asha | 88.0
Sorted by name:
101 | Asha | 88.0
102 | Ravi | 72.0
Students with marks >= 75:
101 | Asha | 88.0
```

Note: In a production application, validate duplicate IDs, empty names, negative marks, and marks above the allowed maximum. A HashMap replaces an existing value if the same key is inserted again.

Day 4 — Task Checklist

■ Practise arrays and common String methods.

■ Add and retrieve records using ArrayList and HashMap.

■ Search by ID and name.

■ Sort records by name or marks.

■ Handle invalid input with try-catch.

■ Read and write a small text file.

■ Use a stream with filter(), sorted(), and forEach().

Quick Revision

Array: fixed-size indexed values. **String:** immutable text. **ArrayList:** resizable ordered list. **HashMap:** key-value lookup. **Exception handling:** manage errors with try-catch. **File handling:** read/write files. **Streams:** process data with operations like filter, map, and sorted.